UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

Technical Report

documentation of the results and interpretation of the robot's behavior

Course: 270130 - ROBOTICS

Professor: JOSEP FERNÀNDEZ RUZAFA

Lab Group: 12

Berta Casas Riera

Arnau Garcia Parise

Table of Contents

Timed Square Path **2**
Objective 2
Explanation of how the code works 2
Results 5
Odometry-Based Square Path **6**
Objective 6
Explanation of how the code works 6
Results 8
LiDAR Data Analysis: **10**
Objective 10
Explanation of how the code works 10
- The Setup and Connection 10
- Gathering the Data 10
- Processing and Mathematics 10

Results 11

Timed Square Path

Objective

Move forward 1 meter, turn 90 degrees, and complete a full square trajectory (returning to the initial pose) using open-loop control based on movement velocity and duration.

Explanation of how the code works (SquareTrajectory_time.py)

```
class SquareTrajectory(Node):
def __init__(self):
   super().__init__('square_node')
   qos_profile_r = QoSProfile(reliability=ReliabilityPolicy.RELIABLE,
depth=10)
   self.publisher = self.create_publisher(TwistStamped, '/cmd_vel',
qos_profile_r)
   self.timer_period = 0.1
   self.timer = self.create_timer(self.timer_period,
self.timer_callback)
```

First of all, we declared a function called SquareTrajectory which represents the ROS node that will control the robot's movement. This class inherits from Node, which means it will have all the functionalities of a ROS2 node, such as publishing messages, using timers, or writing messages to the console. In this class, all the necessary logic for the robot to move in a square pattern will be implemented. This part is the constructor of the class, which is executed when the node is created. First, the node is initialized with the name square_node.

Then, we set up a quality of service profile to ensure that messages are reliable. Then we created a publisher so that it will send velocity messages to the /cmd_vel topic, which is what many robots use to receive movement commands. Finally, a timer is created that is executed every 0.1 seconds and will call the timer_callback function, which contains the movement logic.

```
self.state = 0
self.lados_completados = 0
self.linear_speed = 0.2
self.angular_speed = 0.5
self.target_distance = 1.0
self.target_angle = math.pi / 2
```

In this section, several variables are defined to control the behavior of the robot. state indicates if the robot is moving forward or turning, while lados_completados counts how many sides of the square have already been completed. The robot's speeds are also defined by linear_speed for moving forward and angular_speed for turning. Moreover, the movement goals are defined which are a distance of 1 meter for each side of the square and a turn of 90 degrees at each corner.

```
self.forward_time = self.target_distance / self.linear_speed
self.turn_time = self.target_angle / self.angular_speed
```

```
self.start_time = self.get_clock().now()
self.get_logger().info('Iniciando cuadrado: Avanzando lado 1...')
```

This part calculates how long each movement should last. To do this, it uses the relationship between distance, speed, and time. The time to move forward is calculated by dividing the side distance by the linear speed, while the time to turn is calculated by dividing the angle by the angular speed. The moment when the movement starts is also stored with start_time, which will be used to calculate the elapsed time. Finally, a message is displayed on the console indicating that the robot is starting on the first side of the square.

```
def timer_callback(self):
        if self.lados_completados >= 4:
            self.stop_robot()
            if self.timer is not None:
                self.get_logger().info('¡Cuadrado de 1x1m completado
con éxito!')
                self.timer.cancel()
                self.timer = None
            return
        current_time = self.get_clock().now()
        elapsed_time = (current_time - self.start_time).nanoseconds /
1e9
        move_msg = TwistStamped()
        move_msg.header.stamp = self.get_clock().now().to_msg()
        move_msg.header.frame_id = ''
```

This part runs every time the timer is activated. Here it constantly checks what the robot should do according to the current state. First, it verifies if the four sides of the square have already been completed; if so, the robot stops and the timer is canceled. If the square has not yet been finished, the time elapsed since the current movement began is calculated, and a speed message is created that will be sent to the robot.

```
if self.state == 0:
            if elapsed_time < self.forward_time:
                move_msg.twist.linear.x = self.linear_speed
                move_msg.twist.angular.z = 0.0
            else:
                self.state = 1
                self.start_time = self.get_clock().now()
                self.get_logger().info('Girando 90 grados...')
```

When the state is 0, the robot is in the advancing phase. This is when the robot moves forward with a defined linear speed and without turning. To confirm that the robot has moved one meter, the program checks if the necessary time to cover the meter of distance has not yet elapsed. If there is still time, the robot continues moving forward. When this time runs out, the program changes the state so that the robot moves on to the turning phase.

```
elif self.state == 1:
            if elapsed_time < self.turn_time:
                move_msg.twist.linear.x = 0.0
```

```
                move_msg.twist.angular.z = self.angular_speed
            else:
                self.state = 0 # Cambiamos al estado Avanzar
                self.lados_completados += 1
                self.start_time = self.get_clock().now()
                if self.lados_completados < 4:
                    self.get_logger().info(f'Lado
{self.lados_completados + 1} comenzando. Avanzando...')
```

With this part of the code we want to make the robot do a 90-degree turn. To do that, whne the state is 1, the robot is in the turning phase, that is when the robot stops moving forward and only applies an angular velocity to turn on itself. This turn continues until the time necessary to make a 90-degree turn. When this time ends, the program changes the state back to moving forward and increments the counter of completed sides.

```
self.publisher.publish(move_msg)
def stop_robot(self):
   move_msg = TwistStamped()
   self.publisher.publish(move_msg)
```

The first line sends the movement message to the /cmd_vel topic. Each time the timer is executed, a new message is published with the speed that the robot should have at that moment. In this way, the robot constantly receives updated commands to move forward or turn depending on the point of the square it is in. The next function is used to stop the robot from moving. To do this, we create a message which is created with no movement value, which means that all speeds are zero. When this message is published, the robot stops moving.

```
def main(args=None):
   print("Iniciando nodo de trayectoria cuadrada...")
   rclpy.init(args=args)
   node = SquareTrajectory()
```

This part is basically the main function that starts the program which displays a message indicating that the node is starting. Then it initializes the ROS2 system with the code line rclpy.init() and creates an instance of the SquareTrajectory class ,which is the one that controls the robot.

```
try:
   rclpy.spin(node)
except KeyboardInterrupt:
   pass
finally:
   node.stop_robot()
   node.destroy_node()
   rclpy.shutdown()
```

First, we maintain the fact that the robot is running by using the line rclpy.spin(node), which allows the timer and the rest of the functionalities to keep working. If the user interrupts the program, the exception is caught and the program continues with the finally block. In this

final block, the robot is stopped, the node is destroyed, and the ROS2 system is safely shut down.

Results

After implementing and running our code, we managed to get the robot to move correctly following the programmed trajectory. The node created in ROS2 sends commands to the robot to move forward a certain distance and then make a 90-degree turn. This process is repeated four times, causing the robot to end up drawing a square with its movement. Unfortunately, the robot does not make a full square due to variables that are out of our control, such as hardware or the floor on which we put the robot.

Odometry-Based Square Path

Objective

Move forward 1 meter, turn 90 degrees, and complete a full square trajectory (returning to the initial pose) using the closed-loop feedback provided by the TurtleBot3 odometry (topic /odom).

Explanation of how the code works (SquareTrajectory_odom.py)

```
def obtener_yaw(q):
   siny_cosp = 2 * (q.w * q.z + q.x * q.y)
   cosy_cosp = 1 - 2 * (q.y * q.y + q.z * q.z)
   return math.atan2(siny_cosp, cosy_cosp)
```

First, this function converts the robot's orientation from a quaternion representation into a yaw angle, which represents the robot's rotation around the vertical axis (Z axis). The function receives a quaternion as input, applies the mathematical formulas to extract the yaw component, and returns the result in radians using the atan2 function. This value is later used to determine how much the robot has rotated.

```
class CuadradoSimple(Node):
def __init__(self):
   super().__init__('cuadrado_simple_node')
```

The __init__ method is executed when the node is created. The call to super().__init__() initializes the ROS node and assigns it the name cuadrado_simple_node. This name identifies the node within the ROS network so that it can communicate with other nodes.

```
self.publicador = self.create_publisher(TwistStamped, '/cmd_vel', 10)
self.suscriptor = self.create_subscription(Odometry, '/odom',
self.odom_callback, 10)
self.timer = self.create_timer(0.033, self.timer_callback)
self.metros_lado = 1.5
self.radianes_giro = (math.pi / 2) * 0.95
self.vel_lineal = 0.2
self.vel_angular = 0.5
self.estado = 0
self.lados_hechos = 0
self.odom_lista = False
self.x_ini, self.y_ini, self.yaw_ini = 0.0, 0.0, 0.0
self.x_act, self.y_act, self.yaw_act = 0.0, 0.0, 0.0
```

This code snippet initializes the main elements of the node. First, a publisher is created that sends velocity commands to the /cmd_vel topic to control the robot's movement, and a subscriber that receives odometry data from the /odom topic to know its position and orientation. A timer is also defined that executes the control loop every 0.033 seconds. Next,

movement parameters are declared, such as the length of the square's side (1 m), the approximate turning angle of 90°, and the linear and angular speeds. Finally, state variables are created to know if the robot is moving forward or turning, how many sides it has completed, and to store the initial and current position of the robot using odometry data.

```
def odom_callback(self, msg):
self.x_act = msg.pose.pose.position.x
self.y_act = msg.pose.pose.position.y
self.yaw_act = obtener_yaw(msg.pose.pose.orientation)
if not self.odom_lista:
   self.reiniciar_referencias()
   self.odom_lista = True
   self.get_logger().info('Empezando cuadrado.')
def reiniciar_referencias(self):
   self.x_ini = self.x_act
   self.y_ini = self.y_act
   self.yaw_ini = self.yaw_act
```

We used all of the code above to get initial information so that the robot knows how to act accordingly. The first function is executed every time a new odometry message is received to update the robot's current position and orientation based on the data contained in the message. Then, the next few lines until the if, it extracts the robot's current X and Y coordinates from the odometry message and calculates the current yaw angle using the previously defined function. We do that because this information is essential to measure the robot's movement. After that, the condition checks if the node has already received its first odometry message. If not, the robot's current position is stored as the starting reference, and the variable odom_lista is set to True. This ensures that the robot begins the square movement only after valid position data has been received. Once finished, the next function saves the robot's current position and orientation as the starting reference for the next movement. This is called whenever the robot finishes moving forward or finishes a turn.

```
def timer_callback(self):
if self.lados_hechos >= 4:
   self.parar_robot()
   self.get_logger().info('¡Fin!')
   self.timer.cancel()
if self.estado == 0:
distancia = math.sqrt((self.x_act - self.x_ini)**2 + (self.y_act -
self.y_ini)**2)
if distancia < self.metros_lado:
   cmd.twist.linear.x = self.vel_lineal
else:
   self.parar_robot()
   self.estado = 1
   self.reiniciar_referencias()
elif self.estado == 1:
diferencia_angulo = self.yaw_act - self.yaw_ini
while diferencia_angulo > math.pi:
   diferencia_angulo -= 2 * math.pi
if abs(diferencia_angulo) < self.radianes_giro:
```

```
    cmd.twist.angular.z = self.vel_angular
else:
    self.parar_robot()
    self.estado = 0
    self.lados_hechos += 1
    self.reiniciar_referencias()
```

In this part of the code there is the timer_callback function, which is the one that runs repeatedly thanks to the timer and where we control the robot's movement. First, we check if the robot has already completed the four sides of the square, if it does, we make it stop, display a completion message, and cancel the timer so that the program stops running. Then we use the variable estado to know what the robot should do at each moment. When the state is 0, the robot moves forward and we calculate the distance it has traveled by comparing the current position with the initial position using the Pythagorean theorem.
If it has not yet reached the distance defined for each side of the square, it continues moving forward. If it has already reached it, we stop it and we change the state to 1, and save the current orientation as a reference to start the turn. When the state is 1, the robot is turning so we calculate how much it has turned relative to the initial angle. As long as it has not reached the turning angle we have defined (about 90 degrees approximately), it continues turning. When it completes the turn, we stop it again, change the state to move forward, increase the counter of sides done, and save the new position as a reference to start the next side of the square. This process repeats until the robot completes all four sides.

```
self.publicador.publish(cmd)
def parar_robot(self):
    self.publicador.publish(TwistStamped())
def main(args=None):
    rclpy.init(args=args)
    node = CuadradoSimple()
```

The first line publishes the velocity command to the /cmd_vel topic. These commands determine how the robot moves at each moment. After that, the first function sends a velocity command with zero speeds, which immediately stops the robot. Then, the second function starts the program by initializing ROS 2 and creating an instance of the CuadradoSimple node.

```
rclpy.spin(node)
node.parar_robot()
node.destroy_node()
rclpy.shutdown()
```

The first line keeps the node running so it can continuously process callbacks and control the robot.Then, when the program finishes or is interrupted, the robot is stopped, the node is destroyed, and ROS 2 is safely shut down.

Results

The executed code successfully controlled the TurtleBot3, directing it to perform a complete square trajectory using closed-loop odometry feedback. Fulfilling the stated objective, the

robot advanced exactly 1 meter per side. To achieve this, the system continuously calculated the Euclidean distance traversed. The node systematically cycled through its internal state machine, alternating between linear translation (State 0) and angular rotation (State 1) until all four sides of the square were completed.

A critical factor in the precision of this maneuver was the deliberate tuning of the rotation angle. Commanding a physical robot to turn exactly 90 degrees often results in overshooting due to the mechanical inertia of the motors. By applying a 0.95 multiplier to the target angle, the code effectively forced the robot to trigger the stop command slightly early. This allowed the robot's natural momentum to carry it precisely into a perfect 90-degree alignment before beginning the next straight segment.

However, because odometry data is derived from wheel encoders, minor mechanical imperfections such as wheel slippage and floor friction inevitably accumulated over the 4-meter journey. As a result, when the node reached the four-side limit and executed the final stop command, the robot's physical resting position exhibited a slight, expected drift and did not perfectly overlap with its absolute starting coordinates.

LiDAR Data Analysis:

Objective

Analyze the information provided by the LiDAR sensor. Perform N (e.g., N=10) consecutive readings from the /scan topic while the robot is stationary and the environment remains unchanged. For each captured angle, calculate and display the minimum, maximum, and average distance values.

Explanation of how the code works (LIDAR_scan.py)

A custom ROS 2 node (analisis_laser_node) was developed to subscribe to the /scan topic using a BEST_EFFORT QoS profile. The robot was placed in a static environment. The node captured 10 full 360-degree arrays of distance data (ranges). Once the target of 10 scans was reached, the system calculated the minimum, maximum, and average distances for each specific angle (from 0° to 359°).

- ### The Setup and Connection

This setup phase establishes the connection to the robot by creating a subscription to the /scan topic via self.create_subscription(...). To handle the high-speed data from the LiDAR, it uses a QoSProfile configured with ReliabilityPolicy.BEST_EFFORT. This specific profile ensures that if a data packet is dropped over the network, ROS 2 simply skips it and waits for the newest one, preventing lag. During this initialization, the node also prepares an empty list called self.historial_scans to store the incoming data arrays and sets the target limit using the variable self.n_scans_objetivo = 10.

- ### Gathering the Data

The core of the data collection happens inside the laser_callback(self, msg) function, which ROS 2 triggers automatically every time a new LaserScan message arrives. First, the callback acts as a gatekeeper: it checks the self.analisis_terminado boolean flag to ensure it only processes data if the analysis is still ongoing. If the flag is false, it takes the massive array of distance measurements, accessed via msg.ranges, and saves it directly into our storage list using self.historial_scans.append(msg.ranges).

The function continuously monitors its progress. Once the condition len(self.historial_scans) == self.n_scans_objetivo is met, it stops gathering data. It then immediately calls the self.calcular_estadisticas() method to start the math and flips self.analisis_terminado to True so no further incoming scans are processed.

- ### Processing and Mathematics

Inside the calcular_estadisticas(self) method, the script shifts from listening to calculating. It first determines the total number of laser beams per scan by checking the length of the first stored array with num_angulos = len(self.historial_scans[0]).

The code then uses a for angulo in range(num_angulos) loop to iterate through every single degree. For each specific angle, it isolates the 10 captured readings using a Python list comprehension: mediciones_angulo = [scan[angulo] for scan in self.historial_scans].

Because sensors frequently return infinity when the laser shoots into open space, attempting to calculate an average with these values would crash the program. To prevent this, the script creates a sanitized list called validas, which strictly filters out both float('inf') and float('-inf'). Finally, if the validas list contains real numbers, the script computes val_min, val_max, and val_media using Python's built-in min(), max(), and sum() / len() functions, printing the formatted results to the terminal.

Results

The output result is below the code (LIDAR_scan.py):

The processed data yielded exactly 360 statistical summaries. The theoretical hardware limits of the sensor are reported as:

- Range Min: ~0.12 meters
- Range Max: 3.50 meters

However, the empirical data revealed three distinct behavioral categories in the sensor's readings:

1. Stable Readings with Variance (Normal Operation): e.g., Angle 000° (Min: 2.103m | Max: 2.133m | Avg: 2.124m).
2. Out-of-Bounds/Reflective Readings: e.g., Angle 020° (Avg: 3.903m).
3. Zero/Invalid Readings: e.g., Angles 036°- 044°, 159°- 176° (Min/Max/Avg: 0.000m).

Understanding these results requires looking beyond the raw statistics to see how the LiDAR physically interacts with its environment. The following section explains the underlying hardware limitations and environmental factors, specifically the test setup in front of a monitor, that caused the observed anomalies and distance variations.

- Inherent Noise

The sensor shows slight distance fluctuations, such as a variance of approximately 3 cm, even when the robot is completely static. This inherent electrical and optical noise proves that raw data must be filtered using techniques like moving averages before being reliably used for navigation.

- Invalid Data and Blind Spots (0.000m)

Readings of 0.000m indicate invalid data or blind spots. Notably, this test was conducted directly in front of a PC monitor. Computer screens are highly problematic for LiDAR sensors because their glass or plastic surfaces cause specular reflection. Instead of returning to the sensor, the laser bounces away like a mirror or is absorbed by the screen's filters. As a result, the receiver gets no return signal, generating these continuous zero readings.

- Max Range Exceeded

The sensor occasionally reported distances up to 3.9m, exceeding its official 3.5m limit. In this context, it is highly likely that the laser experienced a multi-path effect. The beam likely bounced off the smooth monitor screen at an angle, hit a distant surface in the room, and then returned, tricking the sensor into measuring a much longer, distorted distance.